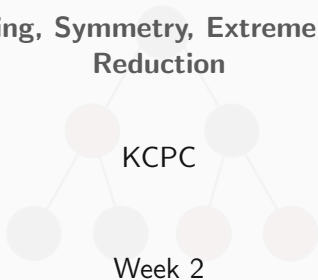


Mathematical Problem Solving II

Wishful Thinking, Symmetry, Extreme Principle, and
Reduction

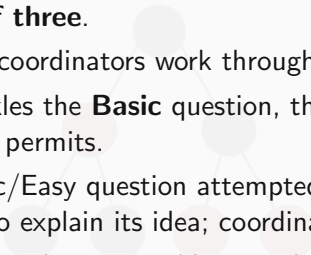


Overview

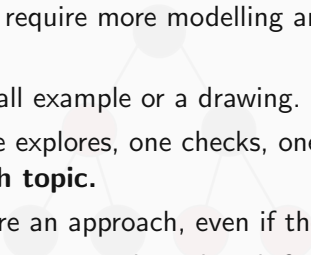
- 1 Problems and Materials
- 2 Week 1 Recap
- 3 Heuristic Methods
- 4 Wishful Thinking
- 5 Symmetry
- 6 Extreme Principle
- 7 Reduction
- 8 Group Problem Lab



Today's Flow: Think, Discuss, Explain

- 
- 1 Form **groups of three**.
 - 2 For each topic, coordinators work through an example.
 - 3 Your group tackles the **Basic** question, then the **Easy** question if time permits.
 - 4 After each Basic/Easy question attempted, we randomly select a group to explain its idea; coordinators fill gaps.
 - 5 Finish with assigned group problems and short presentations.

A Step Up From Workshop 1A

- 
- These problems require more modelling and proof than last week.
 - Start with a small example or a drawing. Say what you tried.
 - In your trio: one explores, one checks, one explains. **Rotate roles after each topic.**
 - Be ready to share an approach, even if the proof is unfinished.
 - Harder questions are stretch work; ask for a hint when stuck.

Problems and Materials



Problems and Materials

Open **Week 2 Problems.pdf** now
and keep it ready for the trio
checkpoints.

Problems and materials on GitHub:

[https://github.com/orthant-kcpc/
kcpc-workshops/tree/main/KCPC%20Year%
202%20Workshops/Pair%201%20-%
20Mathematical%20Problem%20Solving/W2](https://github.com/orthant-kcpc/kcpc-workshops/tree/main/KCPC%20Year%202%20Workshops/Pair%201%20-%20Mathematical%20Problem%20Solving/W2)



Week 1 Recap



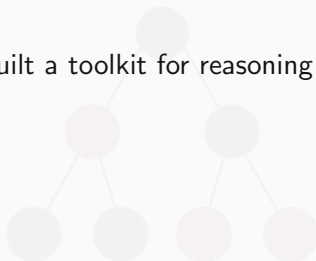
Week 1 Recap

Week 1 Recap



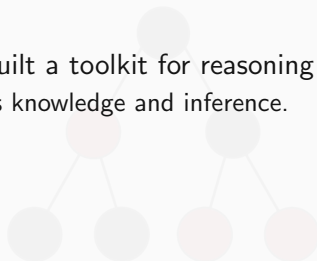
Week 1 Recap

→ Last week we built a toolkit for reasoning about puzzles.



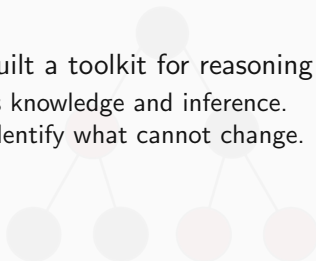
Week 1 Recap

- Last week we built a toolkit for reasoning about puzzles.
- Logic tracks knowledge and inference.

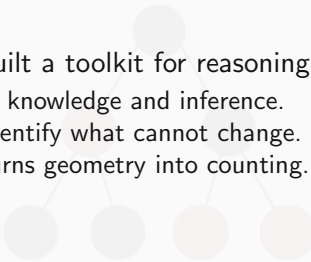


Week 1 Recap

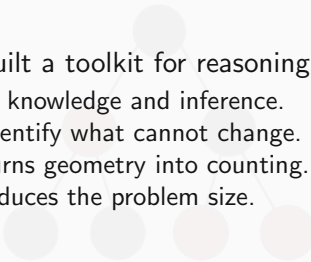
- Last week we built a toolkit for reasoning about puzzles.
 - Logic tracks knowledge and inference.
 - Invariants identify what cannot change.



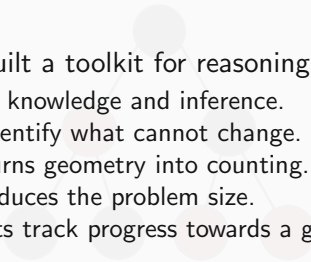
Week 1 Recap

- 
- Last week we built a toolkit for reasoning about puzzles.
 - Logic tracks knowledge and inference.
 - Invariants identify what cannot change.
 - Colouring turns geometry into counting.

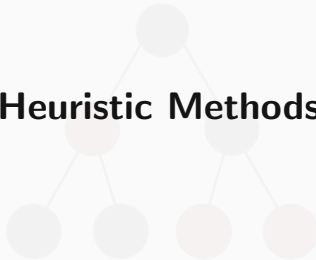
Week 1 Recap

- 
- Last week we built a toolkit for reasoning about puzzles.
 - Logic tracks knowledge and inference.
 - Invariants identify what cannot change.
 - Colouring turns geometry into counting.
 - Induction reduces the problem size.

Week 1 Recap

- 
- Last week we built a toolkit for reasoning about puzzles.
 - Logic tracks knowledge and inference.
 - Invariants identify what cannot change.
 - Colouring turns geometry into counting.
 - Induction reduces the problem size.
 - Monovariants track progress towards a goal.

Heuristic Methods



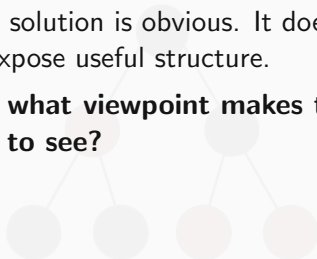
What are Heuristics?



What are Heuristics?

A **heuristic** is a strategy or rule of thumb that helps you choose what to try when no solution is obvious. It does not guarantee success, but it can expose useful structure.

The key question is: **what viewpoint makes the structure of the problem easier to see?**



What are Heuristics?

A **heuristic** is a strategy or rule of thumb that helps you choose what to try when no solution is obvious. It does not guarantee success, but it can expose useful structure.

The key question is: **what viewpoint makes the structure of the problem easier to see?**

→ **Wishful thinking:** imagine a simpler or nicer form.

What are Heuristics?

A **heuristic** is a strategy or rule of thumb that helps you choose what to try when no solution is obvious. It does not guarantee success, but it can expose useful structure.

The key question is: **what viewpoint makes the structure of the problem easier to see?**

- **Wishful thinking:** imagine a simpler or nicer form.
- **Symmetry:** pair objects or equivalent cases.

What are Heuristics?

A **heuristic** is a strategy or rule of thumb that helps you choose what to try when no solution is obvious. It does not guarantee success, but it can expose useful structure.

The key question is: **what viewpoint makes the structure of the problem easier to see?**

- **Wishful thinking:** imagine a simpler or nicer form.
- **Symmetry:** pair objects or equivalent cases.
- **Extreme principle:** inspect a boundary object.

What are Heuristics?

A **heuristic** is a strategy or rule of thumb that helps you choose what to try when no solution is obvious. It does not guarantee success, but it can expose useful structure.

The key question is: **what viewpoint makes the structure of the problem easier to see?**

- **Wishful thinking:** imagine a simpler or nicer form.
- **Symmetry:** pair objects or equivalent cases.
- **Extreme principle:** inspect a boundary object.
- **Reduction:** transform the problem into a familiar one.



Wishful Thinking

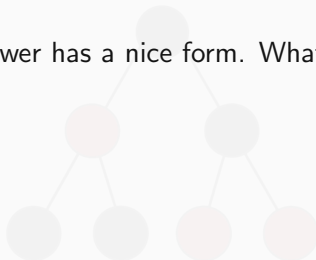
Wishful Thinking

Wishful Thinking: Simplify Until Solvable



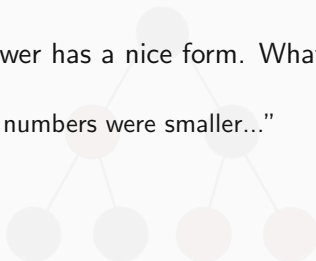
Wishful Thinking: Simplify Until Solvable

→ Assume the answer has a nice form. What would make this easy?

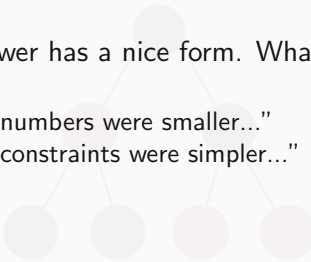


Wishful Thinking: Simplify Until Solvable

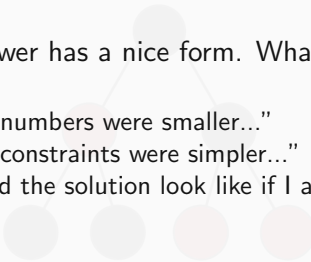
- Assume the answer has a nice form. What would make this easy?
- “If only the numbers were smaller...”



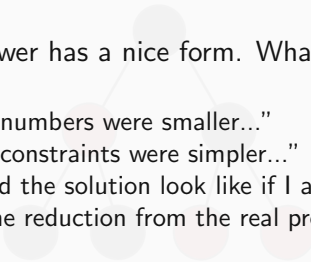
Wishful Thinking: Simplify Until Solvable

- 
- Assume the answer has a nice form. What would make this easy?
 - “If only the numbers were smaller...”
 - “If only the constraints were simpler...”

Wishful Thinking: Simplify Until Solvable

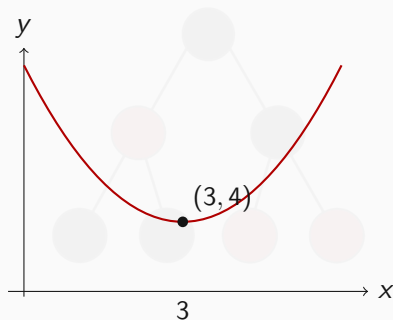
- 
- Assume the answer has a nice form. What would make this easy?
 - “If only the numbers were smaller...”
 - “If only the constraints were simpler...”
 - “What would the solution look like if I already knew it?”

Wishful Thinking: Simplify Until Solvable

- 
- Assume the answer has a nice form. What would make this easy?
 - “If only the numbers were smaller...”
 - “If only the constraints were simpler...”
 - “What would the solution look like if I already knew it?”
 - Then find the reduction from the real problem to the simplified one.

Coordinator Example: Wish for a Square

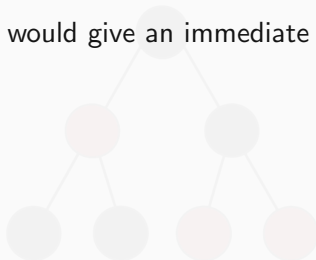
Find the minimum of $x^2 - 6x + 13$ for real x .



The drawing suggests a minimum. How can we **prove** it?

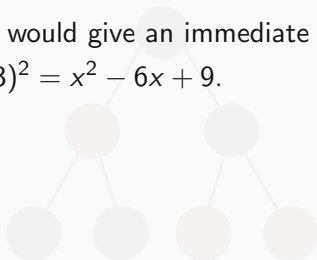
Walkthrough: Complete the Square

→ **Wish:** a square would give an immediate lower bound.



Walkthrough: Complete the Square

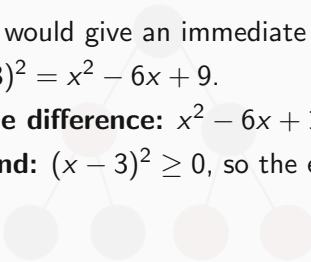
- **Wish:** a square would give an immediate lower bound.
- **Build it:** $(x - 3)^2 = x^2 - 6x + 9$.



Walkthrough: Complete the Square

- **Wish:** a square would give an immediate lower bound.
- **Build it:** $(x - 3)^2 = x^2 - 6x + 9$.
- **Account for the difference:** $x^2 - 6x + 13 = (x - 3)^2 + 4$.

Walkthrough: Complete the Square

- 
- **Wish:** a square would give an immediate lower bound.
 - **Build it:** $(x - 3)^2 = x^2 - 6x + 9$.
 - **Account for the difference:** $x^2 - 6x + 13 = (x - 3)^2 + 4$.
 - **Prove the bound:** $(x - 3)^2 \geq 0$, so the expression is at least 4.

Walkthrough: Complete the Square

- **Wish:** a square would give an immediate lower bound.
- **Build it:** $(x - 3)^2 = x^2 - 6x + 9$.
- **Account for the difference:** $x^2 - 6x + 13 = (x - 3)^2 + 4$.
- **Prove the bound:** $(x - 3)^2 \geq 0$, so the expression is at least 4.
- **Check attainability:** at $x = 3$, its value is 4. Therefore the minimum is exactly 4.

Trio Checkpoint: Wishful Thinking

Basic: Complete the Square

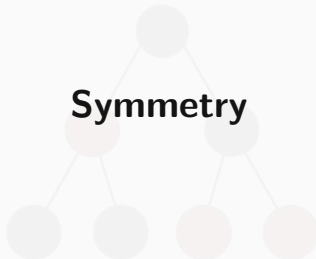
$$x^2 + y^2 \geq \frac{(x + y)^2}{2} \quad (x, y \in \mathbb{R}).$$

Rewrite the difference as a square. When is equality possible?

Easy, if ready: Make the Sum Telescope (problem sheet).

Share: a randomly selected trio explains its rewrite and why it proves the claim. We discuss each question attempted.

Symmetry

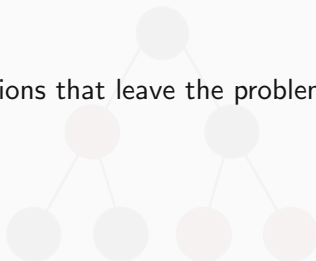


Symmetry: Exploit Invariance Under Transformations



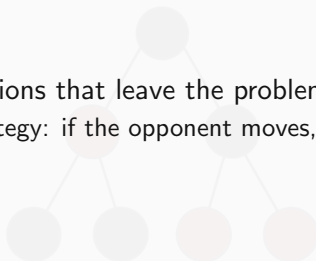
Symmetry: Exploit Invariance Under Transformations

→ Look for operations that leave the problem unchanged.



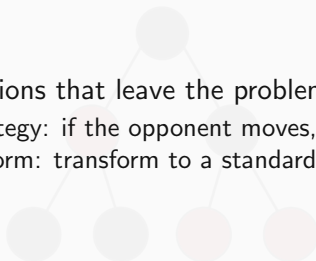
Symmetry: Exploit Invariance Under Transformations

- Look for operations that leave the problem unchanged.
- Pairing strategy: if the opponent moves, reply symmetrically.

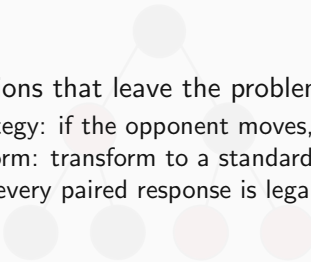


Symmetry: Exploit Invariance Under Transformations

- Look for operations that leave the problem unchanged.
 - Pairing strategy: if the opponent moves, reply symmetrically.
 - Canonical form: transform to a standard representative.



Symmetry: Exploit Invariance Under Transformations

- 
- Look for operations that leave the problem unchanged.
 - Pairing strategy: if the opponent moves, reply symmetrically.
 - Canonical form: transform to a standard representative.
 - Check that every paired response is legal.

Worked Example: Symmetry – Pairing Game

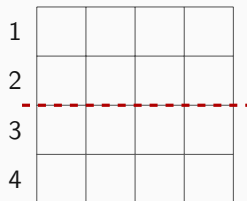
Two players alternately place a **horizontal** domino on a 4×4 board.

Dominoes cannot overlap.

The player who cannot move loses.

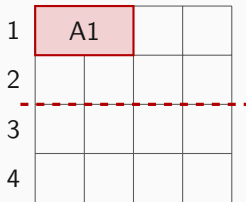
Show the second player has a winning strategy.

What does reflection in the dashed line do to a move?

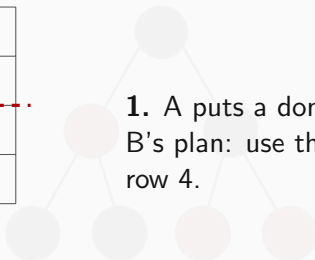


A: first player
B: second player

Pairing Game: Watch Two Rounds

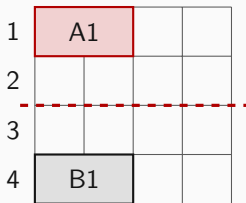


A: first player
B: second player



1. A puts a domino in row 1.
B's plan: use the same columns in row 4.

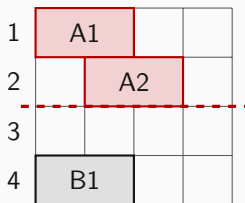
Pairing Game: Watch Two Rounds



A: first player
B: second player

2. B reflects A's move.
The occupied cells are now symmetric.

Pairing Game: Watch Two Rounds



A: first player
B: second player

3. A next plays in row 2.
Which cells should B use?

Pairing Game: Watch Two Rounds

1	A1		
2		A2	
3		B2	
4	B1		

A: first player
B: second player

4. B plays in row 3.
Symmetry is restored after the second round too.

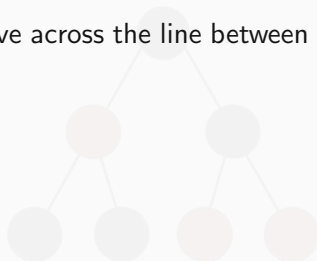
Symmetry

Solution: Pairing Game



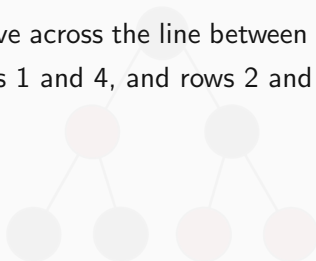
Solution: Pairing Game

→ Mirror each move across the line between the middle two rows.



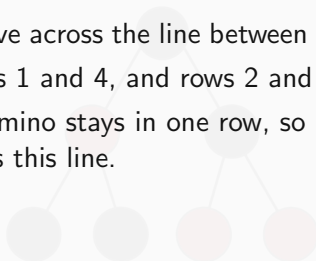
Solution: Pairing Game

- Mirror each move across the line between the middle two rows.
- This swaps rows 1 and 4, and rows 2 and 3.

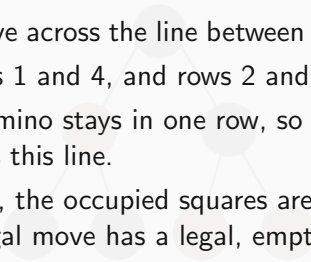


Solution: Pairing Game

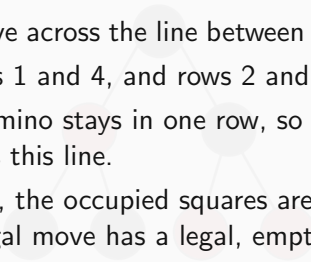
- Mirror each move across the line between the middle two rows.
- This swaps rows 1 and 4, and rows 2 and 3.
- A horizontal domino stays in one row, so it never overlaps its reflection across this line.



Solution: Pairing Game

- 
- Mirror each move across the line between the middle two rows.
 - This swaps rows 1 and 4, and rows 2 and 3.
 - A horizontal domino stays in one row, so it never overlaps its reflection across this line.
 - After each reply, the occupied squares are symmetric; the first player's next legal move has a legal, empty mirror.

Solution: Pairing Game

- 
- Mirror each move across the line between the middle two rows.
 - This swaps rows 1 and 4, and rows 2 and 3.
 - A horizontal domino stays in one row, so it never overlaps its reflection across this line.
 - After each reply, the occupied squares are symmetric; the first player's next legal move has a legal, empty mirror.
 - The second player never runs out of moves first, so wins.

Trio Checkpoint: Symmetry

Basic: Pairing Game

Generalise our strategy to a $2r \times c$ board, with $r \geq 1$, $c \geq 2$, using horizontal dominoes only.

Explain: which rows pair up? Why is the reply always empty and disjoint from the new domino?

Easy, if ready: Coins on a Circular Table. What should happen at the centre?

Share: randomly selected trios explain each attempted question; coordinators complete the legality argument.

Extreme Principle



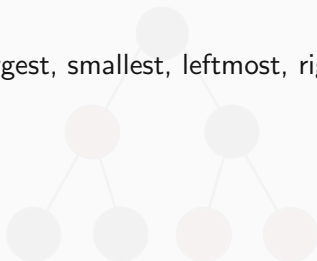
Extreme Principle

Extreme Principle: The Boundary Holds The Answer



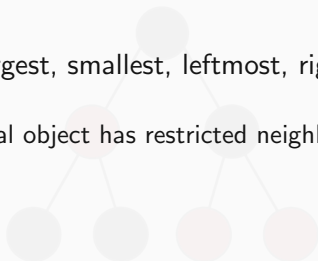
Extreme Principle: The Boundary Holds The Answer

- Consider the largest, smallest, leftmost, rightmost, or most extreme object.



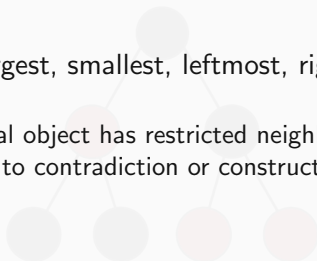
Extreme Principle: The Boundary Holds The Answer

- Consider the largest, smallest, leftmost, rightmost, or most extreme object.
 - The extremal object has restricted neighbours.

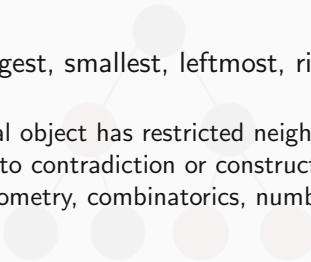


Extreme Principle: The Boundary Holds The Answer

- Consider the largest, smallest, leftmost, rightmost, or most extreme object.
 - The extremal object has restricted neighbours.
 - Often leads to contradiction or constructive greedy choice.

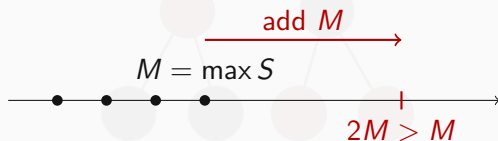


Extreme Principle: The Boundary Holds The Answer

- 
- Consider the largest, smallest, leftmost, rightmost, or most extreme object.
 - The extremal object has restricted neighbours.
 - Often leads to contradiction or constructive greedy choice.
 - Works in geometry, combinatorics, number theory, and algorithms.

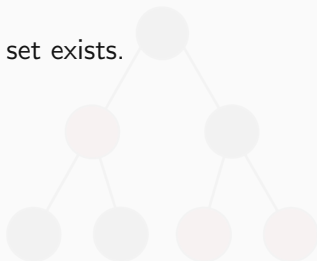
Coordinator Example: A Largest Element

Could a finite, non-empty set of **positive** integers be closed under addition? That means $a + b \in S$ for all $a, b \in S$, including $a = b$.



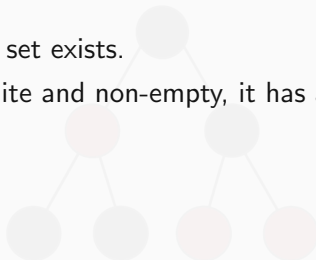
Solution: The Maximum Cannot Grow

→ Suppose such a set exists.



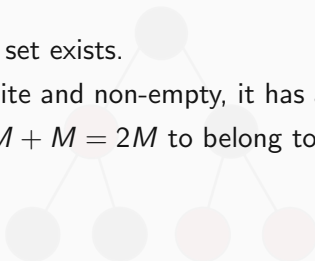
Solution: The Maximum Cannot Grow

- Suppose such a set exists.
- Because it is finite and non-empty, it has a largest element M .



Solution: The Maximum Cannot Grow

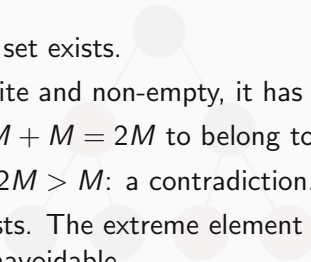
- Suppose such a set exists.
- Because it is finite and non-empty, it has a largest element M .
- Closure forces $M + M = 2M$ to belong to the set.



Solution: The Maximum Cannot Grow

- Suppose such a set exists.
- Because it is finite and non-empty, it has a largest element M .
- Closure forces $M + M = 2M$ to belong to the set.
- Positivity gives $2M > M$: a contradiction.

Solution: The Maximum Cannot Grow

- 
- Suppose such a set exists.
 - Because it is finite and non-empty, it has a largest element M .
 - Closure forces $M + M = 2M$ to belong to the set.
 - Positivity gives $2M > M$: a contradiction.
 - No such set exists. The extreme element made the contradiction unavoidable.

Trio Checkpoint: Extreme Principle

Basic: A Finite Set Closed Under Addition

Now allow **all integers**, including negative numbers and zero. If S is finite, non-empty, and closed under addition, prove $S = \{0\}$.

Explain: what does the maximum tell you? What extra information comes from the minimum?

Easy, if ready: Minimum Degree Forces a Cycle. Use an endpoint of a longest path.

Share: random trios explain each attempted question.

Reduction



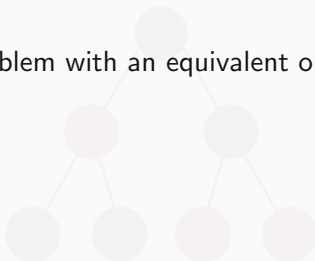
Reduction

Reduction: Change The Representation



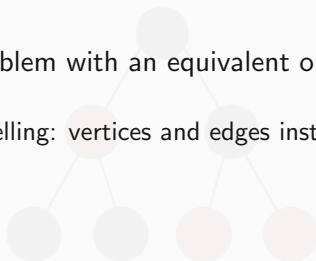
Reduction: Change The Representation

- Replace the problem with an equivalent one that is easier to see.



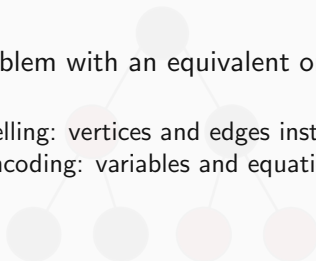
Reduction: Change The Representation

- Replace the problem with an equivalent one that is easier to see.
 - Graph modelling: vertices and edges instead of story.

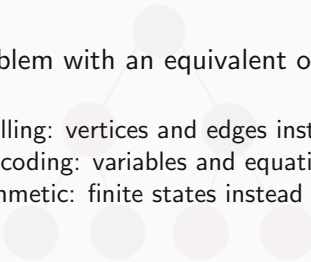


Reduction: Change The Representation

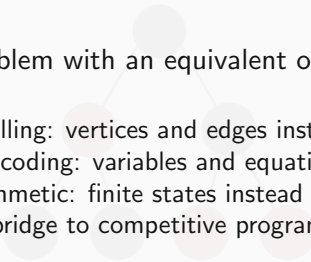
- Replace the problem with an equivalent one that is easier to see.
 - Graph modelling: vertices and edges instead of story.
 - Algebraic encoding: variables and equations instead of logic.



Reduction: Change The Representation

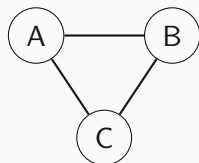
- 
- Replace the problem with an equivalent one that is easier to see.
 - Graph modelling: vertices and edges instead of story.
 - Algebraic encoding: variables and equations instead of logic.
 - Modulo arithmetic: finite states instead of infinite.

Reduction: Change The Representation

- 
- Replace the problem with an equivalent one that is easier to see.
 - Graph modelling: vertices and edges instead of story.
 - Algebraic encoding: variables and equations instead of logic.
 - Modulo arithmetic: finite states instead of infinite.
 - This is the bridge to competitive programming.

Graph Vocabulary for Reduction

- A **vertex** represents an object or a complete puzzle state.
- An **edge** represents a relationship or a legal move.
- A vertex's **degree** counts its incident edges.
- A **path** follows edges without repeating vertices; a **cycle** closes back at its start.

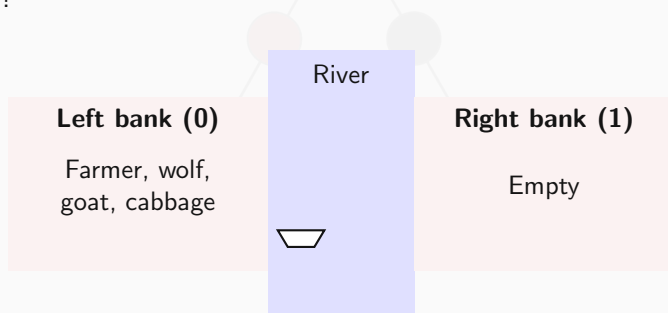


Each degree is 2.

Worked Example: Reduction – River Crossing

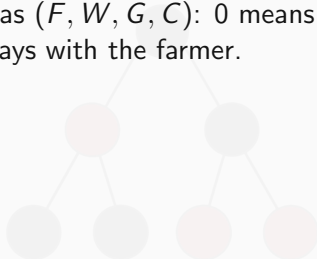
A farmer must take a wolf, a goat, and a cabbage across a river. The boat carries only the farmer and at most one item.

The wolf cannot be left with the goat, and the goat cannot be left with the cabbage **without the farmer**. How can everyone cross safely?



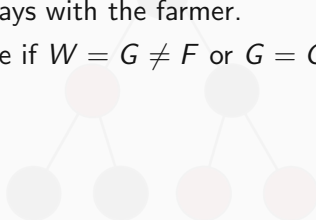
River Crossing: Build the State Model

- Encode a state as (F, W, G, C) : 0 means left, 1 means right.
The boat is always with the farmer.



River Crossing: Build the State Model

- Encode a state as (F, W, G, C) : 0 means left, 1 means right. The boat is always with the farmer.
- A state is unsafe if $W = G \neq F$ or $G = C \neq F$.



River Crossing: Build the State Model

- Encode a state as (F, W, G, C) : 0 means left, 1 means right. The boat is always with the farmer.
- A state is unsafe if $W = G \neq F$ or $G = C \neq F$.
- A legal crossing flips F and, optionally, one item starting on the farmer's bank. The new state must be safe.

River Crossing: Build the State Model

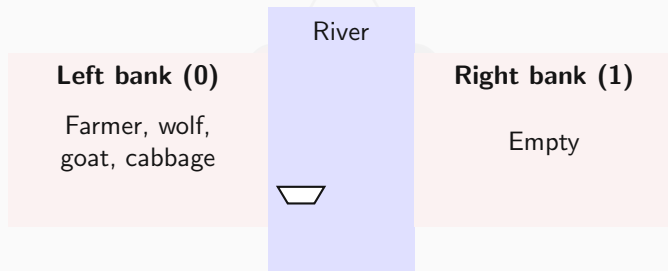
- Encode a state as (F, W, G, C) : 0 means left, 1 means right. The boat is always with the farmer.
- A state is unsafe if $W = G \neq F$ or $G = C \neq F$.
- A legal crossing flips F and, optionally, one item starting on the farmer's bank. The new state must be safe.
- The start is 0000; the goal is 1111.

River Crossing: Build the State Model

- Encode a state as (F, W, G, C) : 0 means left, 1 means right. The boat is always with the farmer.
- A state is unsafe if $W = G \neq F$ or $G = C \neq F$.
- A legal crossing flips F and, optionally, one item starting on the farmer's bank. The new state must be safe.
- The start is 0000; the goal is 1111.
- States are vertices; legal crossings are edges. We need a **path** between start and goal.

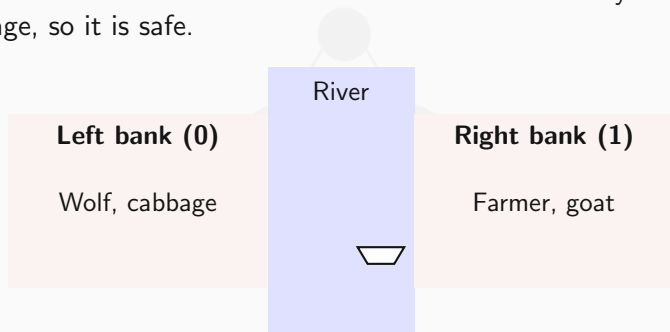
Full Walkthrough: River Crossing (1/2)

Start: 0000. Only the goat can go first: taking the wolf leaves goat/cabbage; taking cabbage leaves wolf/goat.



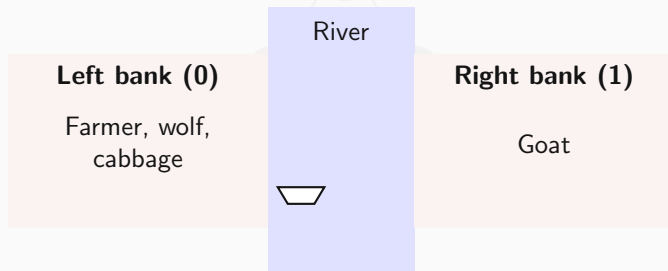
Full Walkthrough: River Crossing (1/2)

1. **Goat over:** 1010. The unattended bank has only wolf and cabbage, so it is safe.



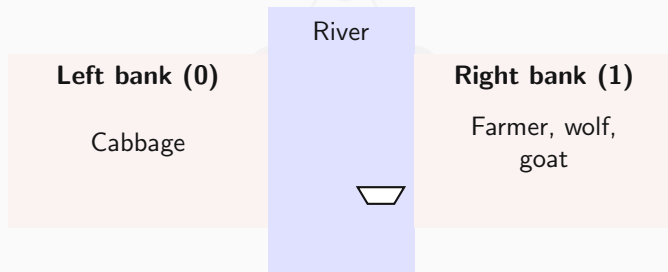
Full Walkthrough: River Crossing (1/2)

2. Farmer back alone: 0010. The goat can wait safely on its own. Bringing it back would undo our progress.



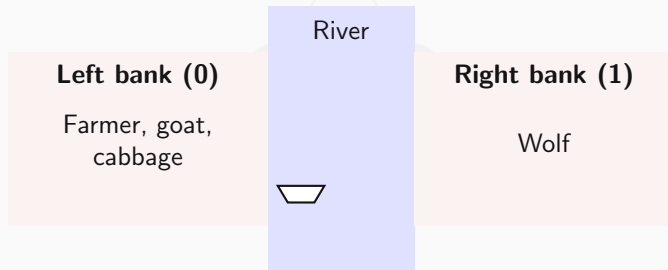
Full Walkthrough: River Crossing (1/2)

3. Wolf over: 1110. Wolf and goat are safe together while the farmer is present. How can the farmer return?



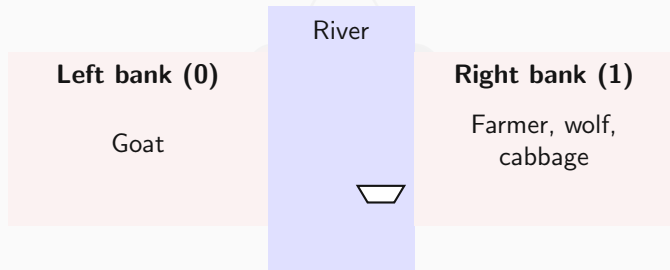
Full Walkthrough: River Crossing (2/2)

4. Goat back: 0100. The farmer cannot leave wolf and goat alone. Bringing the goat back makes both banks safe.



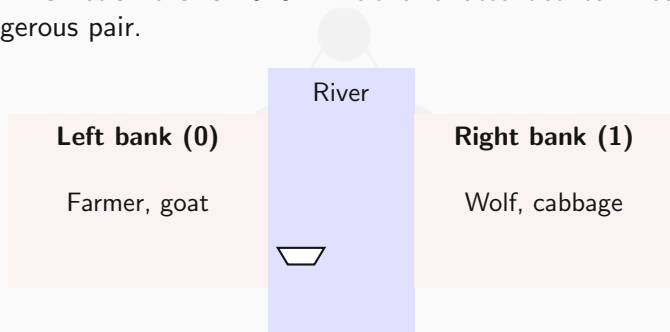
Full Walkthrough: River Crossing (2/2)

5. Cabbage over: 1101. Wolf and cabbage can be left together; the goat waits alone on the left.



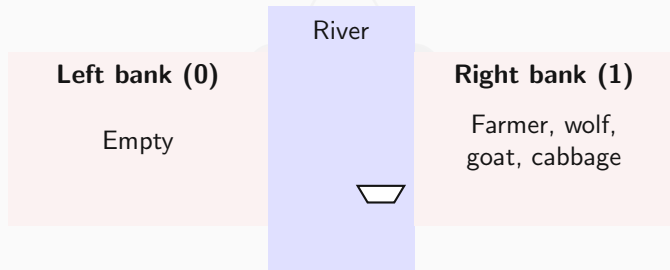
Full Walkthrough: River Crossing (2/2)

6. Farmer back alone: 0101. Neither unattended bank contains a dangerous pair.

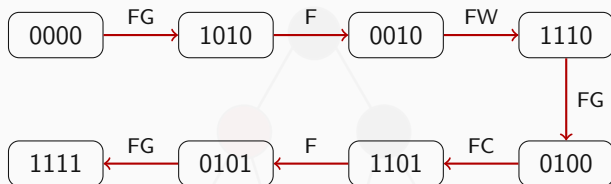


Full Walkthrough: River Crossing (2/2)

7. Goat over: 1111. Everyone is on the right bank. Every crossing was legal and every intermediate state safe.



River Crossing: A Path Through Safe States



Edge labels show who crosses: F = farmer, W = wolf, G = goat, C = cabbage. This is **one solution path**, not the whole state graph.

Takeaway: model the rules first, then search the legal states.

Trio Checkpoint: Reduction

Basic: An Acquaintance Graph

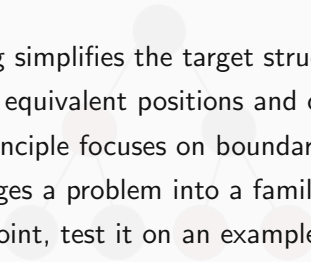
At a party of $n \geq 2$ people, acquaintance is mutual. Prove that two people have the same number of acquaintances.

Try: draw a small example, then list the possible degrees. Can degree 0 and degree $n - 1$ occur together?

Easy, if ready: The Bridges of Königsberg.

Share: random trios explain each attempted question; coordinators complete the counting argument if needed.

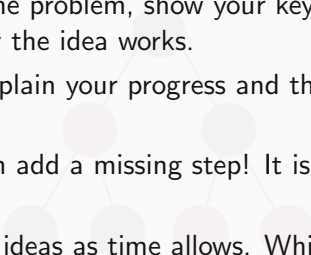
Our Toolkit for Group Problems

- 
- Wishful thinking simplifies the target structure.
 - Symmetry pairs equivalent positions and cases.
 - The extreme principle focuses on boundary objects.
 - Reduction changes a problem into a familiar representation.
 - Choose a viewpoint, test it on an example, then justify it.

Group Problem Lab



Debrief

- 
- In trios: state the problem, show your key drawing or rewrite, and explain why the idea works.
 - If unfinished, explain your progress and the exact sticking point.
 - Another trio can add a missing step! It is supposed to be collaborative.
 - Share follow-up ideas as time allows. Which heuristic helped?
 - Further practice: remaining sheet problems and **Week 2 Advanced Slides**.

Thanks for coming!

See ya next week!

I hope you enjoyed the workshop and found some
useful new ways to approach problems.